

SAÉ 2.01-2.02 : Rapport POO

Versions 1, 2 et 3

Yann Renard
Tom Cox
Alexandre Sorel

6 juin 2026

Table des matières

1 Version 1 : Un seul moyen de transport	3
1.1 Lancement et utilisation	3
1.2 Diagramme UML et mécanismes objets	3
1.3 Analyse technique des fonctionnalités	5
1.4 Analyse des tests	5
2 Version 2 : Multi-modalité	7
2.1 Lancement et utilisation	7
2.2 Diagramme UML et mécanismes objets	7
2.3 Analyse technique des fonctionnalités	8
2.4 Analyse des tests	9
3 Version 3 : Multi-critères et historique	10
3.1 Lancement et utilisation	10
3.2 Diagramme UML et mécanismes objets	10
3.3 Analyse technique des fonctionnalités	13
3.4 Analyse des tests	13

1 Version 1 : Un seul moyen de transport

1.1 Lancement et utilisation

L'application est développée avec Java 21 et Maven. Le code source est disponible sur le dépôt GitLab de l'équipe E5. Pour lancer l'application, il faut d'abord compiler le projet avec Maven, puis exécuter la classe principale.

Compilation et exécution depuis la racine du projet :

```
1 mvn clean exec:java
```

Les fichiers de données CSV doivent être placés dans `src/main/resources/`. La bibliothèque graphes (`sae_s2_2026.jar`, `jgrapht-core-1.5.1.jar`, `jheaps-0.14.jar`) est référencée dans le dossier `lib/` à la racine du projet et déclarée dans le `pom.xml`.

1.2 Diagramme UML et mécanismes objets

Tableau 1 : Classes de la Version 1

Classe/Enum	Package	Rôle
TypeCout	models	Enum représentant les 3 critères : TEMPS, PRIX, CO2
Cout	models	Stocke les 3 valeurs de coût dans une <code>Map<TypeCout, Double></code>
Ville	models	Implémente <code>Lieu</code> (lib IUT). Représente une ville du réseau
Trajet	models	Implémente <code>Connexion</code> (lib IUT). Connexion entre deux <code>Ville</code> avec un <code>Cout</code> et une <code>ModaliteTransport</code>
Voyageur	models	Représente l'utilisateur : nom et <code>TypeCout</code> (critère à optimiser)
Plateforme	models	Classe centrale : contient les listes de <code>Ville</code> et <code>Trajet</code> , expose les fonctionnalités de chargement, filtrage et tri

Table 1.1 : Classes de la Version 1

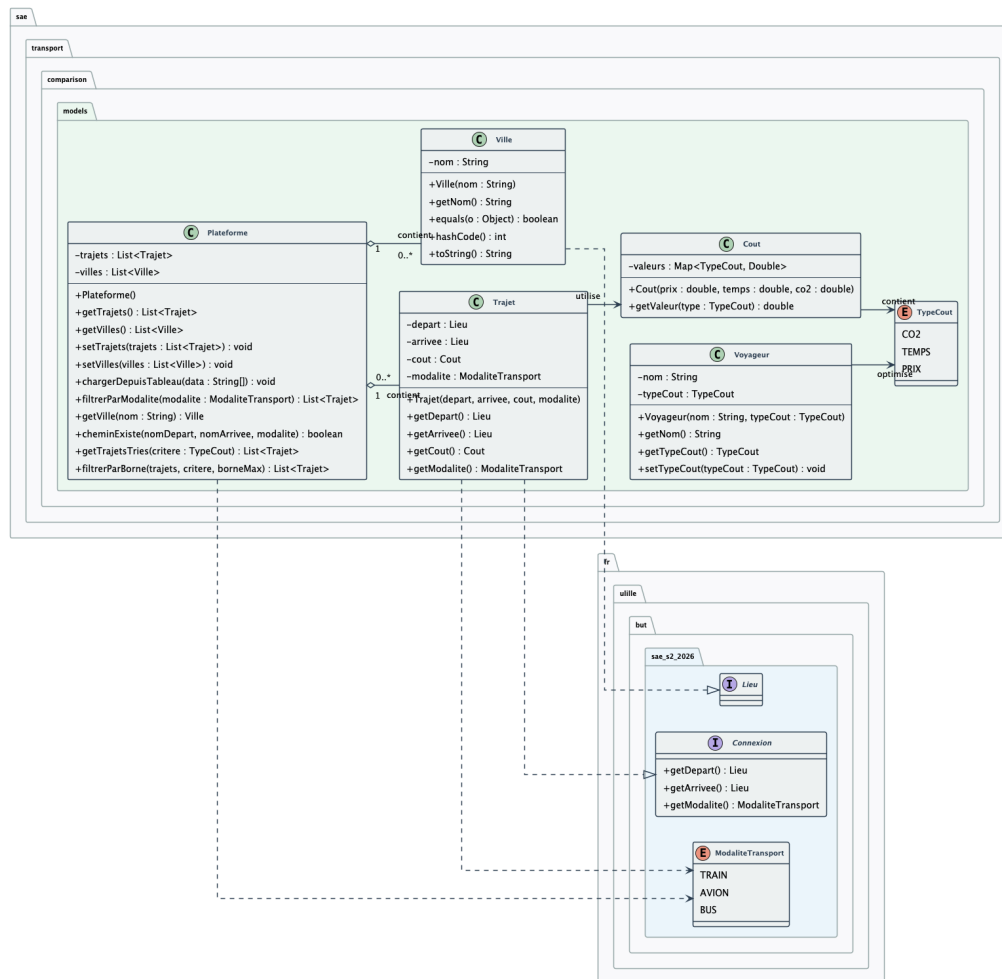


Figure 1.1 : Diagramme UML — Version 1

Mécanismes objets mis en œuvre

Interfaces et implémentation : Les classes `Ville` et `Trajet` implémentent les interfaces `Lieu` et `Connexion` fournies par la bibliothèque de l'IUT. Ce choix permet à notre code d'être directement compatible avec la bibliothèque de graphes, qui attend des objets `Lieu` et `Connexion`. C'est le principe de programmation par interface : on dépend d'une abstraction plutôt que d'une implémentation concrète.

Énumérations : `TypeCout` et `ModaliteTransport` (fournie par la lib) sont des enums. Ce mécanisme est approprié ici car les valeurs possibles sont fixes et connues à l'avance. Utiliser une enum plutôt qu'une `String` évite les erreurs de frappe et rend le code plus lisible.

Collections : La classe `Plateforme` utilise des `List<Ville>` et `List<Trajet>` pour stocker le réseau. La classe `Cout` utilise une `Map<TypeCout, Double>` pour associer chaque type de coût à sa valeur. Ce choix permet d'accéder à n'importe quel critère avec `getValeur(TypeCout)`.

Encapsulation : Tous les attributs sont privés. L'accès se fait uniquement via des getters, ce qui protège l'état interne des objets et permet de modifier l'implémentation sans impacter le code appelant.

1.3 Analyse technique des fonctionnalités

Tableau 2 : Correspondance fonctionnalités / méthodes

Méthode	Fonctionnalité couverte (sujet)
<code>chargerDepuisTableau(String[])</code>	Vérifier la validité des données et reconstituer le réseau
<code>filtrerParModalite(ModaliteTransport)</code>	Filtrer les données sur un moyen de transport spécifique
<code>getVille(String)</code>	Utilitaire interne pour retrouver une ville par son nom
<code>cheminExiste(...)</code>	Déterminer s'il existe un voyage possible entre deux villes
<code>getTrajetsTries(TypeCout)</code>	Afficher les meilleurs voyages ordonnés selon le critère
<code>filtrerParBorne(...)</code>	Exclure les alternatives qui excèdent les bornes définies

Table 1.2 : Correspondance fonctionnalités / méthodes

La méthode `chargerDepuisTableau` effectue la validation des données en vérifiant que chaque ligne contient exactement 6 colonnes et que tous les coûts sont positifs ou nuls. En cas d'erreur, une `IllegalArgumentException` est levée (remplacée par `DonneesInvalidesException` en Version 2).

La recherche des meilleurs itinéraires repose sur la bibliothèque de graphes : on construit un `MultiGrapheOrienteValue` dont les sommets sont les `Ville` et les arêtes les `Trajet` valuées selon le critère choisi, puis on appelle `AlgorithmeKPCC.kpcc` pour obtenir les k plus courts chemins.

1.4 Analyse des tests

Deux classes de test ont été développées avec JUnit 5 : `CoutTest` et `PlateformeTest`.

`CoutTest` couvre :

- La création d'un `Cout` avec des valeurs valides
- La récupération correcte de chaque valeur (PRIX, TEMPS, CO2)

- Le cas limite avec des valeurs à zéro

PlateformeTest couvre :

- Le chargement correct des données : vérification du nombre de villes (4) et de trajets (6)
- La détection des données invalides : ligne incomplète, coûts négatifs
- Le filtrage par modalité : vérification du nombre de trajets TRAIN et AVION
- La recherche de ville : ville existante et ville inexistante (**null**)
- La vérification d'existence de chemin : chemin existant, chemin inexistant, ville inconnue
- Le tri par critère : vérification que le premier élément est bien le moins coûteux
- Le filtrage par borne : vérification que tous les trajets retournés respectent la limite

Ces tests couvrent les cas simples, les cas limites et les cas d'erreur pour chaque fonctionnalité principale de cette version.

2 Version 2 : Multi-modalité

2.1 Lancement et utilisation

Le lancement est identique à la Version 1. Les nouvelles fonctionnalités (import CSV, exceptions, points d'intérêt) s'utilisent directement via la classe `Plateforme`. Les fichiers CSV de test sont disponibles dans `src/test/resources/` :

- `exemple-reseau.csv` : données du réseau au format
`villeDépart;villeArrivée;modalité;prix;co2;durée`
- `exemple-correspondances.csv` : coûts de correspondance au format
`ville;mode1;mode2;durée;co2;prix`

Exemple d'utilisation depuis le code :

```
1 Plateforme p = new Plateforme();  
2 p.chargerDepuisCSV("src/test/resources/exemple-reseau.csv");
```

2.2 Diagramme UML et mécanismes objets

Tableau 3 : Nouveautés de la Version 2

Élément ajouté	Justification (sujet)
<code>AucunCheminException</code>	Signaler l'absence de chemin via un mécanisme d'exception
<code>DonneesInvalidesException</code>	Signaler les problèmes de validité de données via un mécanisme d'exception
<code>chargerDepuisCSV(String)</code>	Import des données issues de fichiers CSV
<code>getPointsInteret(List<Trajet>)</code>	N'afficher que le départ, l'arrivée et les lieux où il y a un changement de modalité

Table 2.1 : Nouveautés de la Version 2

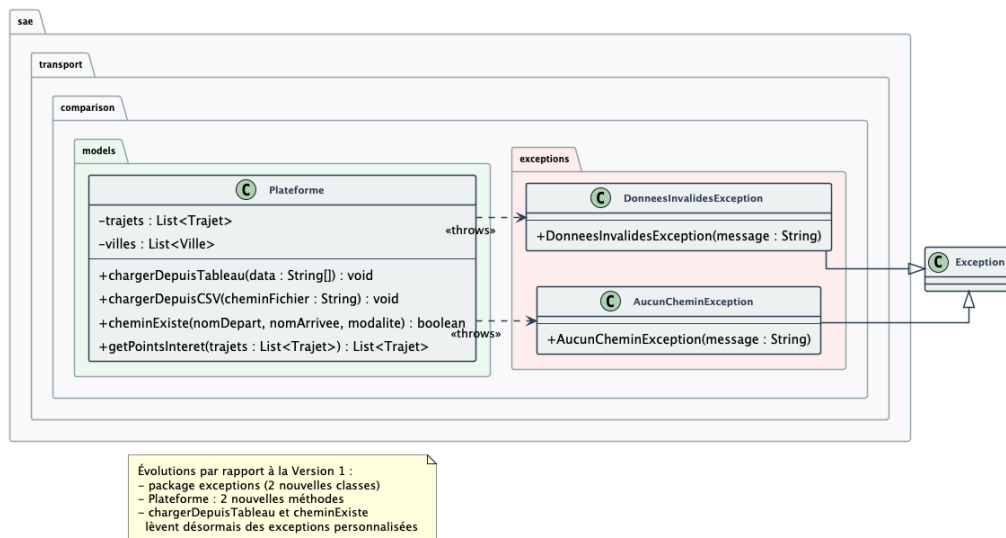


Figure 2.1 : Diagramme UML — Version 2

Mécanismes objets mis en œuvre

Exceptions personnalisées : Deux classes d'exception ont été créées en étendant `Exception`. `AucunCheminException` est levée lorsque les villes demandées sont inconnues ou qu'aucun trajet ne les relie. `DonneesInvalidesException` est levée lorsque les données sont mal formatées ou contiennent des valeurs invalides. Ce mécanisme permet à l'appelant de distinguer les deux types d'erreur et de les traiter séparément.

Restructuration de `cheminExiste` : La méthode ne retourne plus `false` en cas d'absence de chemin : elle lève systématiquement une `AucunCheminException`. Ce choix est plus cohérent avec le principe selon lequel une méthode nommée `cheminExiste` ne devrait jamais retourner `false` si l'appelant s'attend à ce qu'un chemin existe.

Lecture de fichier : La méthode `chargerDepuisCSV` utilise un `BufferedReader` avec `try-with-resources`, ce qui garantit que le fichier est toujours fermé proprement, même en cas d'exception. Les `IOException` sont converties en `DonneesInvalidesException` pour homogénéiser la gestion des erreurs.

2.3 Analyse technique des fonctionnalités

La méthode `getPointsInteret` prend en entrée une liste de `Trajet` représentant un itinéraire complet et retourne uniquement les points significatifs : le premier trajet, le dernier, et ceux dont la modalité diffère du trajet précédent. Cette méthode répond à l'exigence du sujet : « *n'afficher que le départ, l'arrivée et les lieux où il y a un changement dans la modalité de transport* ».

Les exceptions personnalisées améliorent la lisibilité du code appelant : au lieu d'attraper une `IllegalArgumentException` générique, l'appelant peut traiter spécifiquement les erreurs de données invalides et les absences de chemin.

2.4 Analyse des tests

Les classes de test existantes ont été mises à jour pour couvrir les nouvelles fonctionnalités.

`PlateformeTest` — nouveaux cas de test :

- `chargerDepuisCSV` : vérification du chargement depuis un fichier CSV valide
- `cheminExiste` avec exception : vérification que `AucunCheminException` est bien levée pour une ville inconnue et pour deux villes sans connexion directe
- `getPointsInteret` : vérification que seuls le départ, l'arrivée et les changements de mode sont retournés

3 Version 3 : Multi-critères et historique

3.1 Lancement et utilisation

Le lancement reste identique aux versions précédentes. Les nouvelles fonctionnalités sont accessibles directement via les classes du modèle. Le fichier d'historique est un fichier binaire `.ser` généré automatiquement par `HistoriqueManager` dans le répertoire.

Exemple d'utilisation depuis le code :

```
1 // Créer un voyageur avec préférences multi-critères
2 Map<TypeCout, Double> prefs = new EnumMap<>(TypeCout.class);
3 prefs.put(TypeCout.PRIX, 0.5);
4 prefs.put(TypeCout.TEMPS, 0.3);
5 prefs.put(TypeCout.CO2, 0.2);
6 Voyageur voyageur = new Voyageur("Alice", prefs);
7
8 // Obtenir les trajets triés par coût
9 List<Trajet> tries =
10     plateforme.getTrajetsTriesParPreferences(voyageur);
11
12 // Enregistrer un voyage dans l'historique
13 Voyage voyage = new Voyage(tries);
14 voyageur.ajouterVoyage(voyage);
15
16 // Persister l'historique
17 HistoriqueManager manager = new HistoriqueManager("historique.ser");
18 manager.ajouterEtSauvegarder(voyage);
```

3.2 Diagramme UML et mécanismes objets

Tableau 4 : Nouveautés de la Version 3

Élément ajouté / modifié	Justification (sujet)
<code>Voyageur.preferences</code> — <code>Map<TypeCout, Double></code>	Exprimer les préférences entre les critères
<code>Voyageur.historique</code> — <code>List<Voyage></code>	Enregistrer les voyages effectués par l'utilisateur
<code>Voyageur.calculerCoutComposite(Cout)</code>	Adapter le modèle de calcul pour une combinaison de critères
<code>Voyageur</code> implements <code>Serializable</code>	Permettre la sérialisation binaire de l'utilisateur
<code>Voyage</code>	Représenter un voyage effectué avec ses coûts totaux
<code>HistoriqueManager</code>	Gérer la persistance de l'historique via sérialisation binaire
<code>Plateforme.getTrajetsTriesParPreferences(Voyageur)</code>	Trier les trajets selon les préférences de l'utilisateur

Table 3.1 : Nouveautés de la Version 3

Délégation : `HistoriqueManager` est une classe dédiée à la persistance : elle ne connaît pas `Plateforme` ni `Voyageur`, seulement `Voyage`. Ce découplage respecte le principe de responsabilité unique : chaque classe fait une seule chose.

3.3 Analyse technique des fonctionnalités

Préférences multi-critères : Le coût composite d'un trajet est calculé par `calculerCoutComposite` dans `Voyageur` :

```
1 cout composite = poids_PRIX * prix + poids_TEMPS * temps +  
    poids_CO2 * co2
```

La méthode `getTrajetsTriesParPreferences` dans `Plateforme` utilise ce coût composite comme critère de tri, en remplaçant le simple `getValeur(critere)` de la V1 par `voyageur.calculerCoutComposite(trajet.getCout())`. Le tri par sélection reste inchangé.

Historique : `Voyage` encapsule une liste de `Trajet` et expose `getCoutTotal(TypeCout)` qui somme les coûts de tous les trajets. `Voyageur` maintient une `List<Voyage>` en mémoire et expose `getTotalHistorique(TypeCout)` pour calculer le cumul sur tous les voyages : utile pour montrer à l'utilisateur l'évolution de ses habitudes de voyage.

Sérialisation binaire : `HistoriqueManager` utilise `ObjectOutputStream` et `ObjectInputStream` avec try-with-resources pour garantir la fermeture des flux. Si le fichier n'existe pas encore, `charger()` retourne une liste vide plutôt que de lancer une exception. La méthode `ajouterEtSauvegarder` charge l'historique existant, ajoute le nouveau voyage, et réécrit le tout.

3.4 Analyse des tests

Les tests de la Version 3 ont été ajoutés dans `PlateformeTest`.

Tests des préférences multi-critères :

- `getTrajetsTriesParPreferences_retourneTousLesTrajets` : vérification que tous les trajets sont retournés
- `getTrajetsTriesParPreferences_triCroissant` : vérification que le tri est bien croissant sur le coût composite

Tests de l'historique :

- `voyage_coutTotalCorrect` : vérification que `getCoutTotal` somme correctement les coûts
- `voyageur_ajoutVoyageDansHistorique` : vérification qu'un voyage est bien ajouté à l'historique

- `voyageur_totalHistoriqueCumule` : vérification que `getTotalHistorique` cumule correctement sur plusieurs voyages

Tests de la sérialisation :

- `historiqueManager_sauvegardeEtCharge` : vérification que les données sont correctement écrites et relues depuis le fichier binaire
- `historiqueManager_fichierInexistant_retourneListeVide` : vérification du comportement au premier lancement